

Code-Layout, Readability and Reusability

Date	15 July 2026
Team ID	XXXXXX
Project Name	AI Debt Settlement System
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing and indentation used throughout the codebase	Yes	React components and FastAPI files follow consistent formatting
2	Proper File Structure	Files and folders are logically organized	Yes	Separate folders for Components, Pages, Services, Context, Models, Schemas, and API routes
3	Meaningful Variable Names	Variables clearly describe their purpose	Yes	Variables such as loanAmount, monthlyIncome, financialHealthScore, and currentUser improve readability
4	Function / Method Names	Functions are descriptively named	Yes	Functions like fetchLoans(), createAccessToken(), deleteLoan(), and predictSettlement() clearly indicate their functionality
5	Code Comments	Inline and block comments explain the logic	Partial	Important sections are documented, but additional comments can improve maintainability
6	Modular Design	Code is divided into reusable modules and components	Yes	Authentication, Loan Management, Dashboard, Settlement, Profile, and API services are implemented independently

7	No Redundant Code	Duplicate or unused code has been removed	Yes	Shared API service and reusable Layout component reduce code duplication
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Backend uses FastAPI HTTPException; frontend uses try-catch blocks and toast notifications

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Authentication Context (AuthContext)	React Context API	Login, Logout, Protected Routes	High
2	API Service (api.js)	Axios	Dashboard, Loans, Profile, Settlement, Authentication	High
3	ProtectedRoute Component	React Router	All secured pages	High
4	Layout Component	React + Tailwind CSS	Dashboard, Loan Form, Loan History, Settlement, Profile	High
5	Loan Table Component	React	Loan History, Dashboard Summary	Medium
6	JWT Authentication Module	FastAPI + Python	Login, Dashboard, Profile, Loan APIs	High
7	Database Models (models.py)	SQLAlchemy	User Management, Loan Management	High
8	Pydantic Schemas (schemas.py)	FastAPI	Request and Response Validation	High
9	AI Settlement Recommendation Logic	Python	AI Settlement Module	High
10	Dashboard Analytics Module	Python	Dashboard Statistics	Medium
11	Loan CRUD APIs	FastAPI	Add Loan, Edit Loan, Delete Loan, Loan History	High
12	Toast Notification Utility	React Toastify	Login, Registration, Loan Operations	Medium

Overall Code Quality Assessment:

ASPECT	RATIO(1-5)	COMMENTS
--------	------------	----------

Code Layout & Structure	5/5	Project follows a clean MVC architecture with separate folders for frontend, backend, models, schemas, services, and components, making it easy to navigate and maintain.
Readability	5/5	Meaningful variable names, descriptive function names, consistent formatting, and organized React/FastAPI code improve readability and understanding.
Reusability	5/5	Reusable components such as Layout, ProtectedRoute, AuthContext, api.js, and backend modules minimize code duplication and improve maintainability.
Documentation / Comments	4/5	Major modules and API sections are documented with comments. Additional inline comments can further improve code understanding.
Overall Score	4.8 / 5	The project demonstrates a well-structured full-stack implementation with secure authentication, modular architecture, AI-powered debt settlement recommendations, dashboard analytics, and reusable components, making it scalable and maintainable.